FPGA · Basys3

# 분초시계·스톱워치
# RTL 설계와 동작 검증

버튼 입력부터 상태 제어, 시간 계수, FND 표시까지

**발표자 김호철**

개요 → 구현 → 문제 해결 → 배경 기술

Basys3 · Artix-7 · Verilog · Vivado 2026.1

# 두 가지 시간을 하나의 FND에 표시

분초시계는 MM.SS · 스톱워치는 SS.hh (1/100초)

## 분초시계

**12.34**

12분 34초 · 59.59 다음 00.00

## 스톱워치

**12.34**

12.34초 · 99.99 다음 00.00

| btnL · btn[0] | btnC · btn[1] | btnR · btn[2] |
| --- | --- | --- |
| 화면 전환 | 스톱워치 초기화 | 실행 ↔ 일시정지 |

SW15: 전체 리셋 · LED[0/1/2]: CLOCK / RUN / PAUSE

# 개발 환경과 실제 보드

100 MHz 단일 클럭 · 네 자리 7-segment 표시




Basys3 · Artix-7

xc7a35tcpg236-1

Vivado 2026.1 · Verilog · XSim

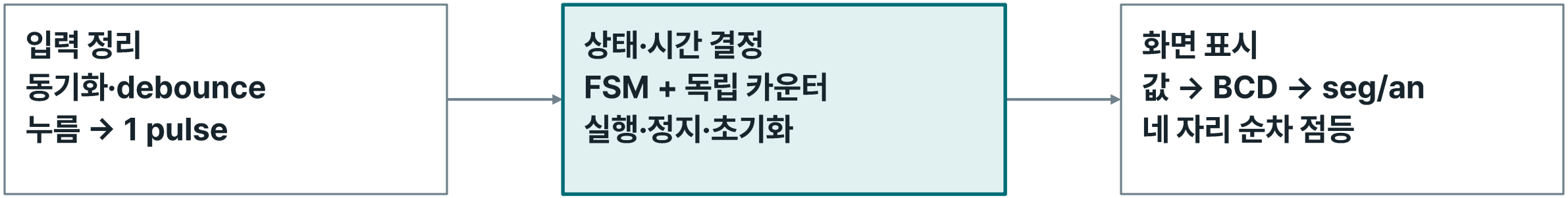
clk: W5 · 100 MHz / 10 ns

btnL: W19 · btnC: U18 · btnR: T17

리셋: SW15(R2) · FND / LED 출력

# 버튼 입력이 FND 화면에 반영되는 과정

입력 정리 → 상태·시간 결정 → 표시 · 모든 모듈은 100 MHz로 병렬 동작




## 동작 예시

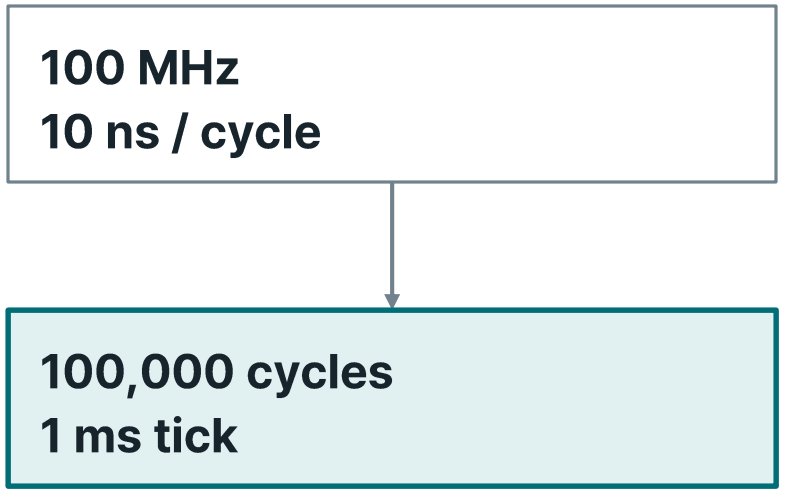
**btnR 누름** → **RUN → PAUSE** → **시간 값 유지**

PAUSE에서 5초 무입력: 숫자 표시를 선분 회전으로 전환

# 1 ms tick을 공통 시간 기준으로 사용

100 MHz 100,000주기마다 · 한 클럭 폭의 enable 펄스

```
else if (r_count == TickCount - 1) begin
  r_count <= 0;
  tick <= 1;
end else begin
  r_count <= r_count + 1;
  tick <= 0;
end
```



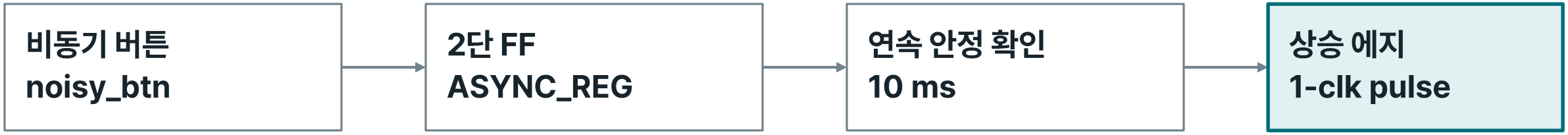

분초시계: 1,000 tick → 1초

스톱워치: 10 tick → 0.01초

FND: 1 tick → 다음 자리

# 버튼 한 번을 한 번의 동작으로

2단 동기화 → 10 ms 연속 안정 확인 → 상승 에지 검출




```
r_prev_btn <= w_clean_btn;
assign w_btn_pulse = w_clean_btn & ~r_prev_btn;
```

## 길게 누르기: clean=1 유지 · pulse는 처음 한 번

리셋: 비동기 assert · 2단 동기 deassert

# FSM은 CLOCK·RUN·PAUSE의 3개 상태

동시 입력 우선순위: btnL > btnC > btnR




btnC: 스톱워치 0 · 현재 상태 유지

전체 reset: CLOCK · 두 카운터 0

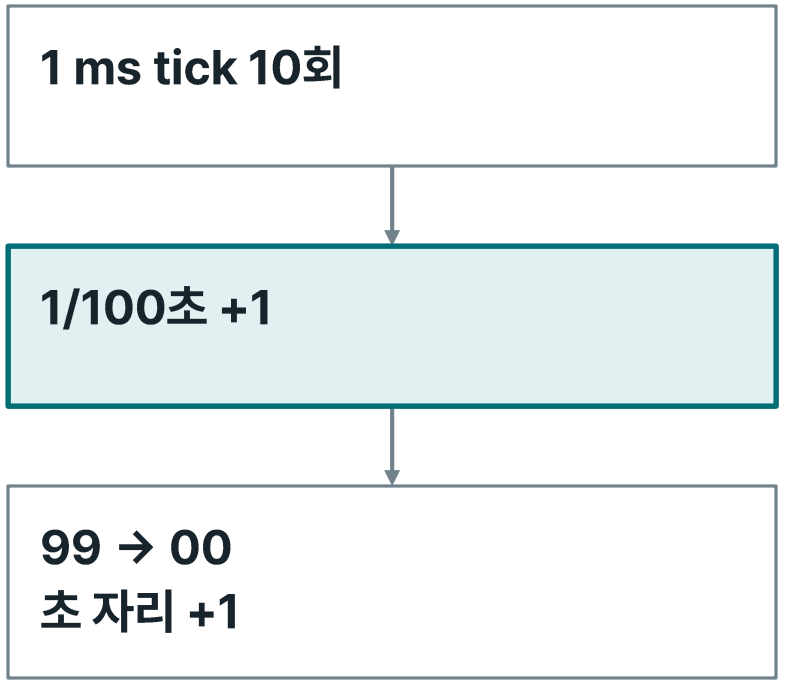
**애니메이션은**
**PAUSE의 화면 효과**

# 스톱워치는 10 tick마다 0.01초 증가

enable=0이면 부분 시간까지 유지 · clear가 계수보다 우선

```
end else if (enable && tick_1ms) begin
  if (r_ms_divider == 9) begin
    r_ms_divider <= 0;
    // 1/100초 증가 및 초 자리 올림
  end else begin
    r_ms_divider <= r_ms_divider + 1;
  end
```

실제 RTL 일부 · 올림 분기 생략




**00.99 → 01.00 · 99.99 → 00.00**

# FND 네 자리를 빠르게 번갈아 켭니다

한 자리 1 ms · 전체 4 ms (250 Hz) · an과 seg는 active LOW

| 일의 자리 | 십의 자리 | 백의 자리 | 천의 자리 |
|---|---|---|---|
| **1110** | **1101** | **1011** | **0111** |
| 0–1 ms | 1–2 ms | 2–3 ms | 3–4 ms |

an[3:0] · 0인 자리만 선택

**display_data**
이진수 → BCD

→

**선택 자리 숫자**
0–9 decode

→

**seg[7:0]**
숫자 + DP

# 일시정지 후 5초 무입력이면 화면 효과

버튼을 누른 동안 대기 시간 0 · 모두 놓은 뒤 다시 측정

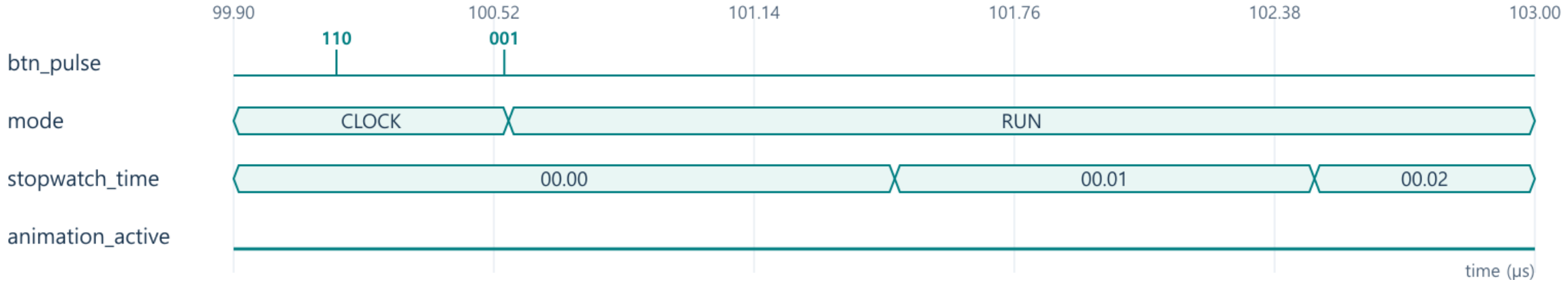



```
assign active = paused && !button_active && (r_idle_count == IdleTimeMs);
```

a → b → c → d → e → f → 반복

버튼 인식: 화면 효과 종료 + 원래 버튼 동작

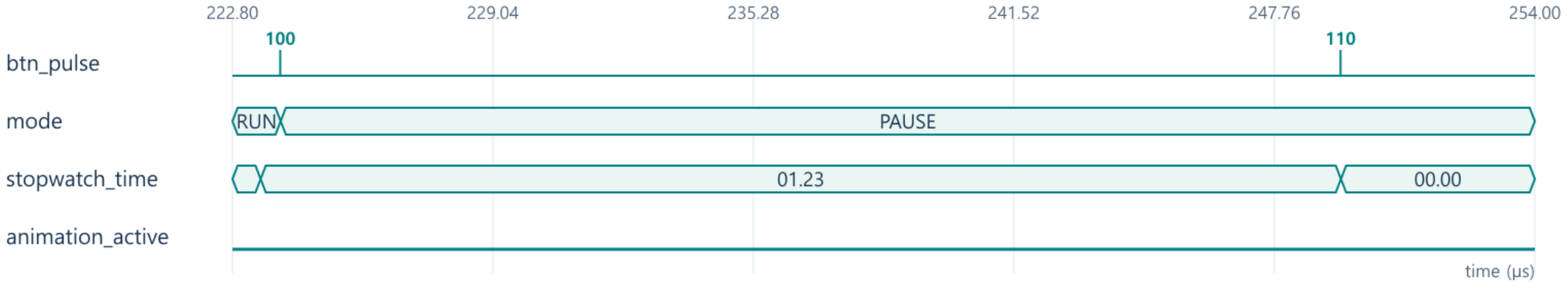
# 파형 ① · btnL 입력 뒤 CLOCK → RUN




**btn_pulse=001 → mode=RUN → 00.00 → 00.01 → 00.02**

**통합 TB: 논리 1 ms = 100 ns · 가로축은 시뮬레이션 시간**

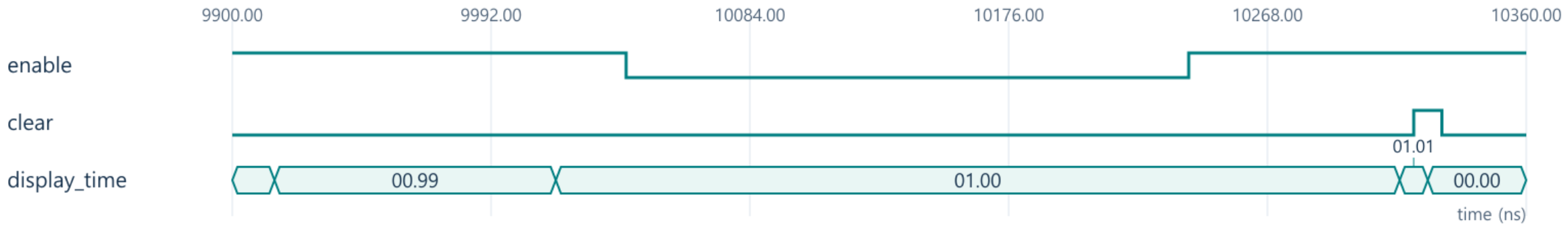
# 파형 ② · PAUSE 유지와 btnC 우선 처리




```
press_button(3'b110);
if (mode !== 2 || stopwatch_time !== 0)
  $fatal(1, "clear must preserve pause");
```

**입력 110 → PAUSE 유지 · 00.00 초기화 · 자동 검사 PASS**

# 카운터 파형 · 올림과 부분 시간 보존




**00.99 → 01.00 · enable=0 동안 정지 · clear로 0**

**카운터 TB: 논리 1 ms tick = 10 ns · 실제 시간과 구분**

통합·카운터 TB 모두 PASS · bitstream 생성 완료

# 실제 보드 동작 시연

영상 클릭으로 재생 · 24.5초 · 저장소 원본 영상 삽입




## 영상에서 확인할 순서

분초시계 → 실행 → 일시정지

5초 무입력 → 선분 회전

재개 → 초기화 → 분초시계 복귀

촬영본: 최종 버튼 연결 수정 이전

최신 수정: Simulation / 구현 검증 완료

# 동시 입력 · 초기화와 상태 전이의 충돌 방지

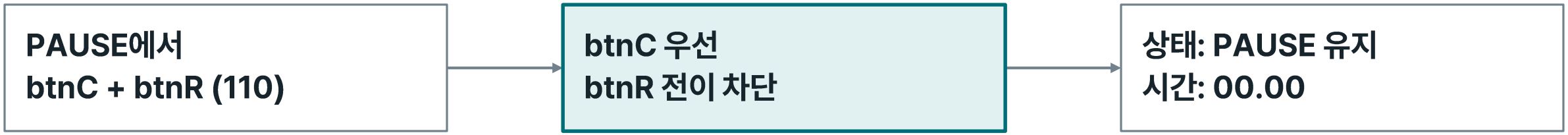
오류 재현 실험 · btnC + btnR 입력에서 btnC 우선 처리

## 재현용 오류 조건

```
if (btn_pulse[2]) begin
```

## 현재 RTL의 조건

```
if (!btn_pulse[1] && btn_pulse[2]) begin
```

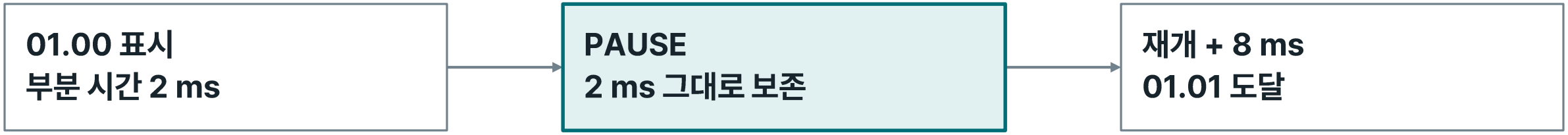



오류 예시: clear must preserve pause → FAIL

현재 RTL: 같은 통합 테스트 → PASS

# Pause / Resume · 부분 계수값까지 보존

오류 재현 실험 · Pause에서 ms 분주값을 지우면 재개 시점이 늦어짐




```
end else if (enable && tick_1ms) begin
  // r_ms_divider / r_centisecond / r_second 갱신
end
```

enable=0: 세 레지스터의 이전 값 유지

분주값을 0으로 지운 오류 예시: resume fraction → FAIL

부분 계수값을 유지하는 현재 RTL: 카운터 TB → PASS

# button_active · 3비트 상태를 한 비트로 축약

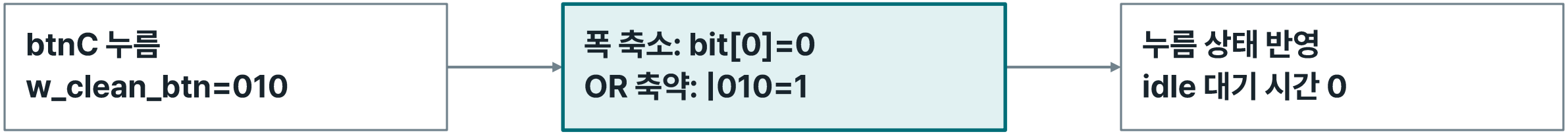
실제 연결 수정 사례 · 버튼 중 하나라도 눌리면 1로 전달

## 이전 연결 / 오류 재현

```
.button_active(w_clean_btn)
```

## 현재 연결

```
.button_active(|w_clean_btn)
```




오류 연결 재현: animation started early → FAIL

현재 RTL: 무입력 경계 · btnC 유지 검사 → PASS

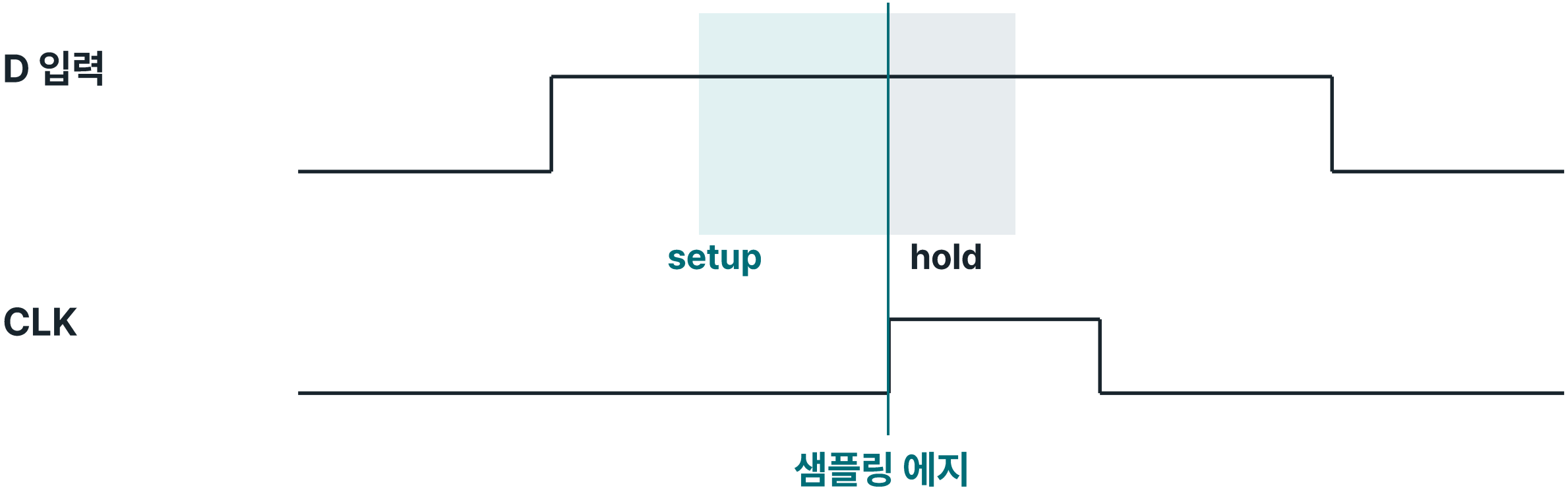
# 구현에서 확인한 세 가지 설계 원칙

기능을 분리하고 · 상태와 표시를 구분하고 · 경계 조건을 검사

| | |
|---|---|
| **독립 카운터** | 분초시계는 계속 계수<br>스톱워치는 부분 시간 보존 |
| **상태·화면 효과** | FSM은 CLOCK / RUN / PAUSE<br>애니메이션은 PAUSE의 출력 효과 |
| **자동 검증** | 동시 입력 · 누른 채 유지<br>올림 · 99.99 순환 · reset |

# Setup / Hold · 샘플링 앞뒤로 입력을 유지

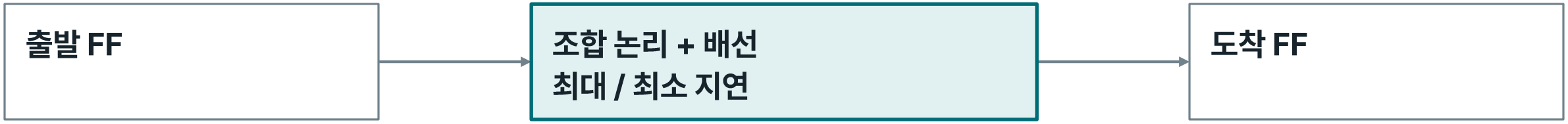
개념도 · 실제 측정 파형 아님 · 클럭 상승 에지에서 D를 저장




← 이 구간에서 D는 같은 값으로 안정 →

# Setup은 너무 늦는 경로 · Hold는 너무 빠른 경로

배치배선 후 최대·최소 지연과 클럭 관계를 함께 검사




## Setup 위반: 늦게 도착

원인: 긴 논리·배선 / 짧은 주기

대응: 경로 축소·pipeline·주기 완화

## Hold 위반: 너무 일찍 도착

원인: 짧은 데이터 경로 / clock skew

대응: 지연 추가·배치배선 조정

**현재 제약 경로: WNS +4.224 ns · WHS +0.122 ns**

# Metastability · 비동기 입력을 동기화하는 이유

2단 동기화는 다음 논리로 불안정성이 전파될 확률을 낮춤



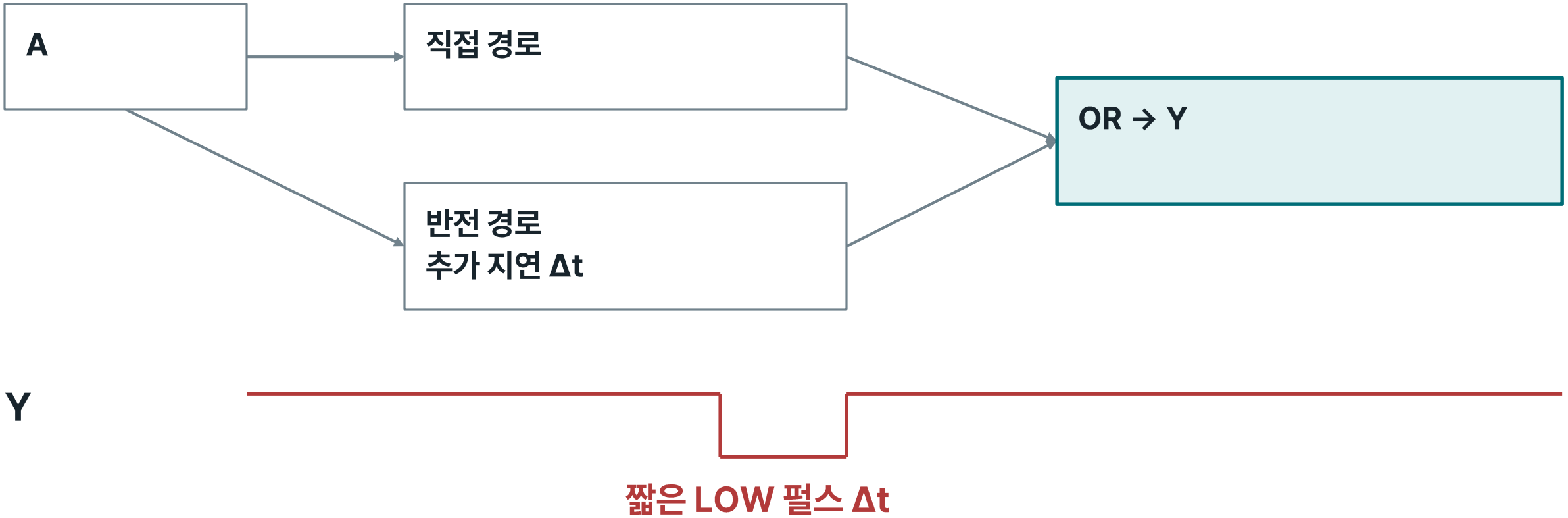

**한 클럭 동안 안정될 시간 확보**

```
(* ASYNC_REG = "TRUE" *) reg r_sync_ff1;
(* ASYNC_REG = "TRUE" *) reg r_sync_ff2;
```

이후 debounce: 새 값 10 ms 연속 유지 확인

# Glitch · 경로 지연 차이가 만드는 짧은 펄스

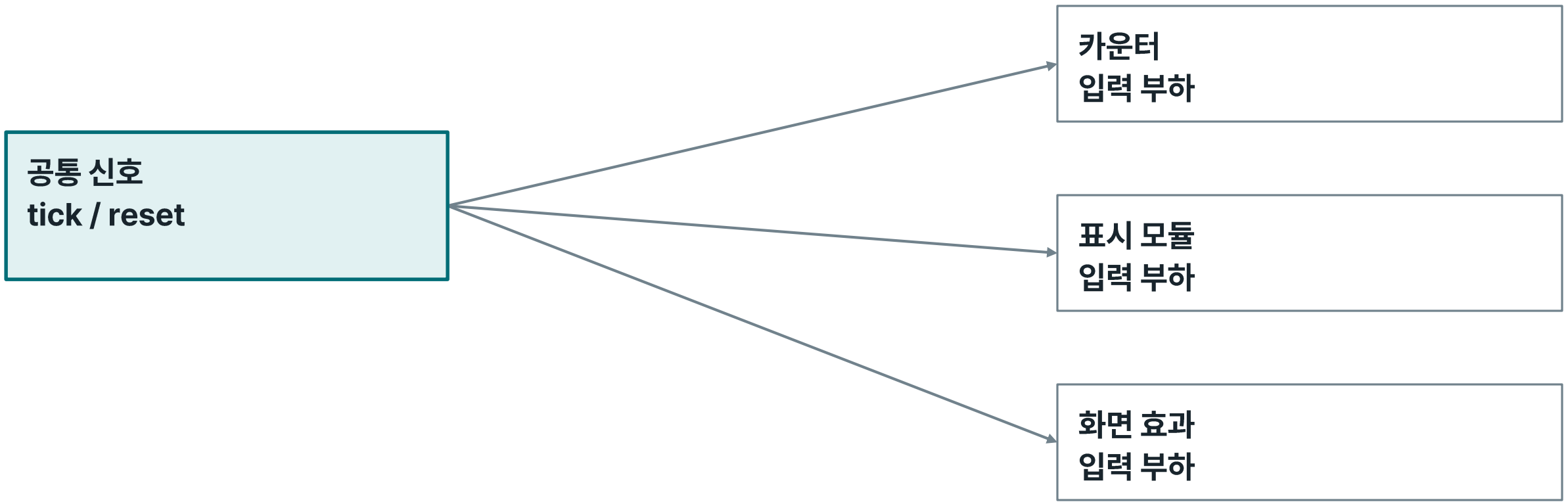
일반 개념 예시 · 논리식은 Y=A OR NOT A이지만 실제 지연은 다름




대응: 출력 register · clock enable · 안정 후 샘플링

# Fan-out · 하나의 신호가 많은 입력을 구동할 때

연결 수 증가 → 부하·배선 증가 가능 → 지연과 타이밍 영향




문제 판단: fan-out 수 + 배선 + slack 보고서

대응: buffer / driver 복제 / 지역별 분배